

SIMATS ENGINEERING

Object Oriented Analysis and Design (CSA1109)

ASSIGNMENT REPORT

Object-Oriented Analysis and UML-Based Design of an Autonomous Drone-Based Disaster Response and Rescue System (ADDRRS)

Field	Details
Course Code & Name	CSA11 – Object Oriented Analysis and Design
Course Outcomes	CO1, CO2, CO3
Bloom's Taxonomy Level	BL3 – Apply, BL4 – Analyse, BL5 – Evaluate, BL6 – Create
SDG Mapping	SDG 3, SDG 9, SDG 11, SDG 13
Assignment Weight age	100%
Student Name / Reg. No.	BILLE LAKESH / 192365037
Date of Submission	03/09/2026

Annexure A – Code of Conduct Declaraton

I **BILLE LAKESH(192365037)**(Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

BILLE LAKESH

1. Problem Analysis and Requirement Summary

1.1 Problem Overview

The Autonomous Drone-Based Disaster Response and Rescue System (ADDRRS) is a software-controlled emergency-management platform that coordinates fleets of autonomous drones to support disaster response operations. The system enables authorised emergency personnel to register disaster incidents, define affected zones, assign drones to survey those zones, receive real-time imagery, video, and environmental sensor data, detect victims, generate alerts, coordinate rescue missions with ground teams, and produce mission-completion reports.

The problem domain combines real-time embedded/autonomous behaviour (drone flight, sensing, communication) with enterprise-style incident and mission management (registration, assignment, reporting), making it a strong candidate for Object-Oriented Analysis and Design because it naturally decomposes into cohesive, loosely-coupled objects: incidents, zones, drones, sensors, missions, teams, victims, alerts, and reports.

1.2 Functional Requirements (derived from the scenario)

- FR1: Authorised users shall authenticate before accessing system functions (role-based access).
- FR2: Emergency Coordinators shall register disaster incidents and define affected zones.
- FR3: Coordinators/Operators shall assign one or more drones to a rescue mission.
- FR4: Drone Operators shall plan and validate flight routes within a zone.
- FR5: Drones shall autonomously navigate, capture images, video, and environmental readings, and transmit them to the command centre.
- FR6: The system shall analyse transmitted sensor data to detect victims and generate alerts/notifications.
- FR7: Coordinators shall create, update, and monitor rescue missions and assign emergency teams.
- FR8: Rescue team members shall update on-ground mission status until completion.
- FR9: The system shall detect drone faults / low battery and trigger failure-handling (emergency landing / return-to-home).
- FR10: The system shall generate a mission-completion report summarising outcomes.
- FR11: Administrators shall manage the drone fleet, user accounts, and shared resources.

1.3 Non-Functional Requirements

- NFR1 – Real-time communication: telemetry and sensor data must reach the command centre with low latency.
- NFR2 – Reliability & fault tolerance: autonomous failure handling must operate without human intervention when links are lost.
- NFR3 – Security: role-based authentication and encrypted communication for command and telemetry links.
- NFR4 – Scalability: the architecture must support many concurrent drones, missions, and zones.

- NFR5 – Maintainability: low coupling / high cohesion so components (sensors, communication, UI) can evolve independently.

1.4 Key Objects Identified in the Scenario

Preliminary noun-phrase analysis of the problem statement yields the following candidate objects, which are refined into classes in Section 5: Disaster Incident, Affected Zone, Drone, Sensor, Sensor Data, Rescue Mission, Victim, Emergency Team, Alert, Resource, Route, Mission Report, and User (Administrator, Emergency Coordinator, Drone Operator, Rescue Team Member).

2. Actor Identification

Actors represent roles that interact with ADDRRS. Both human roles and system/external actors are identified, consistent with the system boundary shown in the Use Case Diagram (Section 4).

S.NO	Actor	Type	Description
1	System Administrator	Primary – Human	Manages user accounts, the drone fleet inventory, and shared resources.
2	Emergency Coordinator	Primary – Human	Registers incidents, defines affected zones, assigns drones/teams, reviews reports.
3	Drone Operator	Primary – Human	Plans flight routes, monitors telemetry, manages fault handling / manual override.
4	Rescue Team Member	Primary – Human	Receives mission dispatch and updates on-ground mission status.
5	Autonomous Drone	Secondary System –	Captures and transmits sensor data; executes navigation and failure protocols autonomously.
6	Weather / GIS Service	Secondary External System –	Supplies weather and terrain/airspace data used during route planning.

3. Use Case Descriptions

The following fully-dressed descriptions specify the primary use cases shown in the Use Case Diagram (Section 4).

UC-01: Register Disaster Incident

Field	Description
Use Case ID / Name	UC-01 – Register Disaster Incident
Actors	Emergency Coordinator
Pre-condition	Coordinator is authenticated.
Main Flow	1. Coordinator logs in. 2. Coordinator selects 'Register Incident'. 3. System presents incident form. 4. Coordinator enters type, severity, location. 5. System creates DisasterIncident and sets status OPEN. 6. System confirms creation.
Alternate / Exception Flow	3a. Duplicate incident detected nearby -> system prompts to merge or escalate existing incident.
Post-condition	A new DisasterIncident exists with status OPEN and is available for zone definition.

UC-02: Assign Drone to Mission

Field	Description
Use Case ID / Name	UC-02 – Assign Drone to Mission
Actors	Emergency Coordinator, Drone Operator (includes Plan Route)
Pre-condition	An AffectedZone has been defined; at least one drone is IDLE and charged.
Main Flow	1. Coordinator selects zone and available drone(s). 2. System includes 'Plan Route' use case. 3. Operator validates/adjusts route. 4. System creates RescueMission linking zone, drone(s) and route. 5. System sets Drone state to PRE FLIGHT.
Alternate / Exception Flow	1a. No drone available -> system notifies Coordinator and suggests waiting/reallocation.
Post-condition	RescueMission is created and drone is scheduled for take-off.

UC-03: Capture and Transmit Sensor Data (includes Detect Victim)

Field	Description
Use Case ID / Name	UC-03 – Capture and Transmit Sensor Data
Actors	Autonomous Drone (system actor)
Pre-condition	Drone state is EN_ROUTE or SURVEYING within an assigned zone.
Main Flow	1. Drone captures image/video/environmental reading via onboard Sensor. 2. Sensor produces SensorData. 3. Drone transmits SensorData to the command centre. 4. System logs the reading and includes 'Detect Victim (AI-assisted)'. 5. On positive detection, system extends to 'Generate Alert / Notification'.
Alternate / Exception Flow	3a. Communication link lost -> data buffered onboard and retried when link restored.
Post-condition	SensorData is logged; victims, if detected, are recorded and alerts raised.

UC-04: Manage Rescue Mission

Field	Description
Use Case ID / Name	UC-04 – Manage Rescue Mission
Actors	Emergency Coordinator (includes Assign Rescue Team, Generate Mission Completion Report)
Pre-condition	A RescueMission exists and at least one victim/objective is identified.
Main Flow	1. Coordinator reviews mission status and detected victims. 2. Coordinator assigns an EmergencyTeam. 3. Rescue team updates mission status as work proceeds. 4. On completion, system includes 'Generate Mission Completion Report'.
Alternate / Exception Flow	2a. No team available -> Coordinator escalates to a neighbouring command centre.
Post-condition	Mission status reaches COMPLETED and a MissionReport is produced.

UC-05: Handle Drone Failure / Emergency Landing

Field	Description
Use Case ID / Name	UC-05 – Handle Drone Failure / Emergency Landing
Actors	Autonomous Drone (system actor), Drone Operator
Pre-condition	Drone is airborne (EN_ROUTE, SURVEYING, or RETURNING).
Main Flow	1. Drone/onboard agent detects a fault or low battery. 2. System extends to 'Generate Alert / Notification' (fault alert). 3. Drone autonomously initiates emergency landing or return-to-home. 4. Operator is notified and may take manual control.
Alternate / Exception Flow	3a. Manual override received from Operator -> Operator directly pilots the drone to a safe landing site.
Post-condition	Drone lands safely (or is grounded for maintenance) and the incident is logged.

4. Use Case Diagram

Figure 1 shows the system boundary of ADDRRS, its actors, and the relationships (<<include>>, <<extend>>) among the identified use cases.

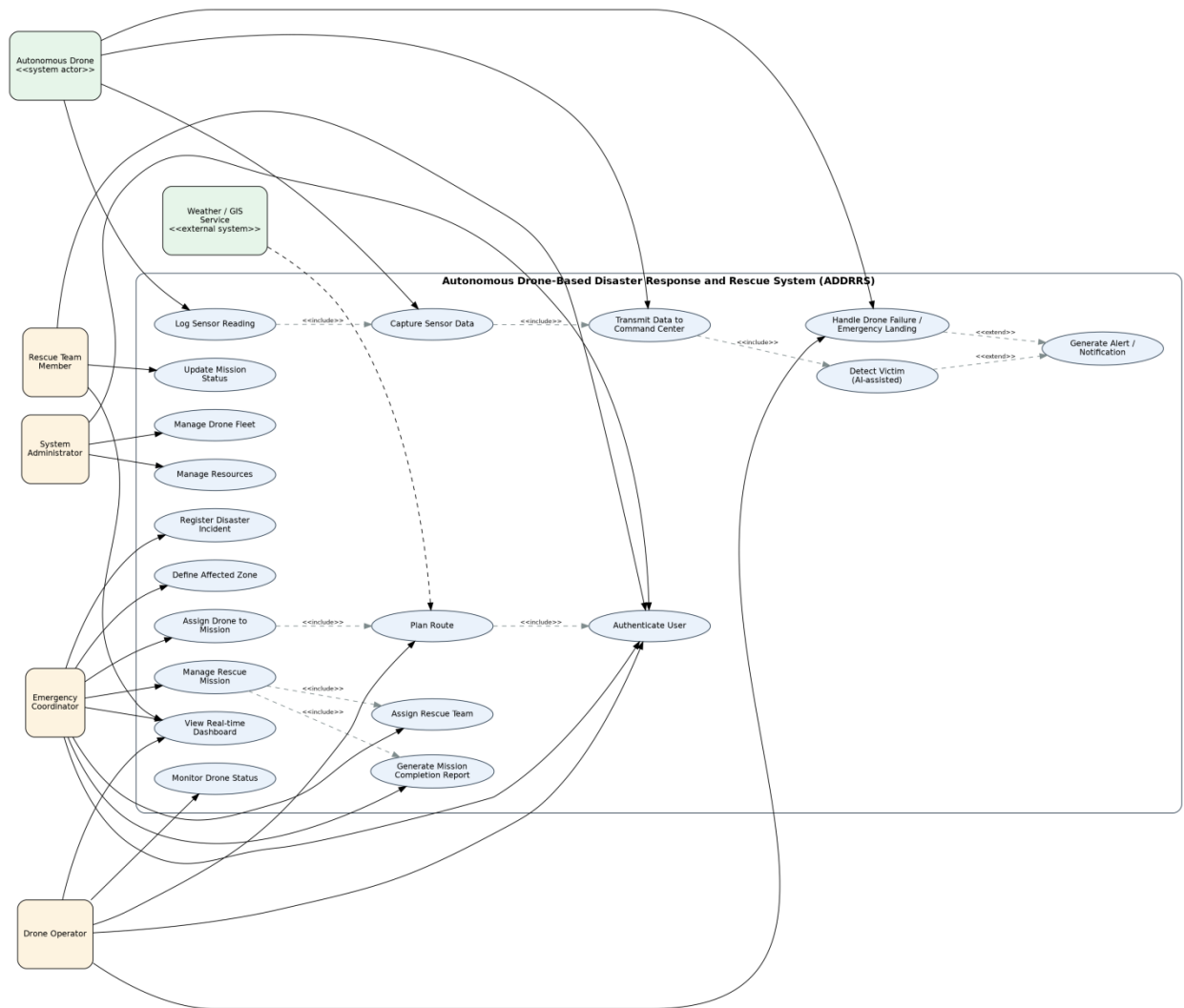


Figure 1: Use Case Diagram for ADDRRS

5. Object and Class Identification

Classes were identified using noun-phrase analysis of the problem statement, refined against CRC responsibilities (Section 13). Each class is shown with representative attributes, methods, and its structural role.

Class	Key Attributes	Key Methods	Responsibility Summary
User (abstract)	userId, name, email, role	login(), logout()	Common identity/authentication behaviour for all human roles.
SystemAdministrator	accessLevel	manageDroneFleet(), manageResources()	Administers accounts, fleet and shared resources.
EmergencyCoordinator	department	registerIncident(), assignDrone()	Owns incident and mission management workflow.
DroneOperator	licenseId	planRoute(), handleFailure()	Plans routes and manages drone telemetry/faults.
RescueTeamMember	skillSet	updateMissionStatus()	Executes on-ground rescue actions and status updates.

DisasterIncident	incidentId, type, severity, status	openIncident(), escalate()	Represents a reported disaster event; owns affected zones.
AffectedZone	zoneId, coordinates, riskLevel	defineBoundary()	Geographic area requiring survey/rescue.
Drone	droneId, batteryLevel, location, state	takeOff(), navigateTo(), land()	Autonomous aerial unit; owns sensors and executes missions.
Sensor (abstract)	sensorId, status	capture(), calibrate()	Generalises onboard sensing capability.
Camera / EnvironmentalSensor	resolution / type	captureImage() / readTemperature()	Concrete sensing devices specialised from Sensor.
SensorData	dataId, timestamp, payloadType, geoTag	transmit(), validate()	Immutable record of a single sensor capture.
RescueMission	missionId, status, priority	assignTeam(), completeMission()	Coordinates drone(s), team, and outcome for a zone.
Victim	victimId, location, condition	updateCondition()	Represents a detected person requiring rescue.
EmergencyTeam	teamId, members	dispatch(), addMember()	Ground rescue unit assigned to a mission.
Alert	alertId, severity, message	notify(), acknowledge()	Notification raised from detections/faults.
Resource	resourceId, type, quantity	allocate(), release()	Consumable/shared assets (medical kits, batteries).
Route	routeId, waypoints, distance	optimize(), validateAirspace()	Planned flight path for a drone.
MissionReport	reportId, summary, victimsRescued	export()	Final summary artefact of a completed mission.

6. Class Diagram

Figure 2 presents the structural (static) design: inheritance (User, Sensor hierarchies), composition (Drone owns Sensor; DisasterIncident owns AffectedZone), aggregation (RescueMission aggregates Drone and EmergencyTeam), and directed associations that realise the requirements in Section 1.

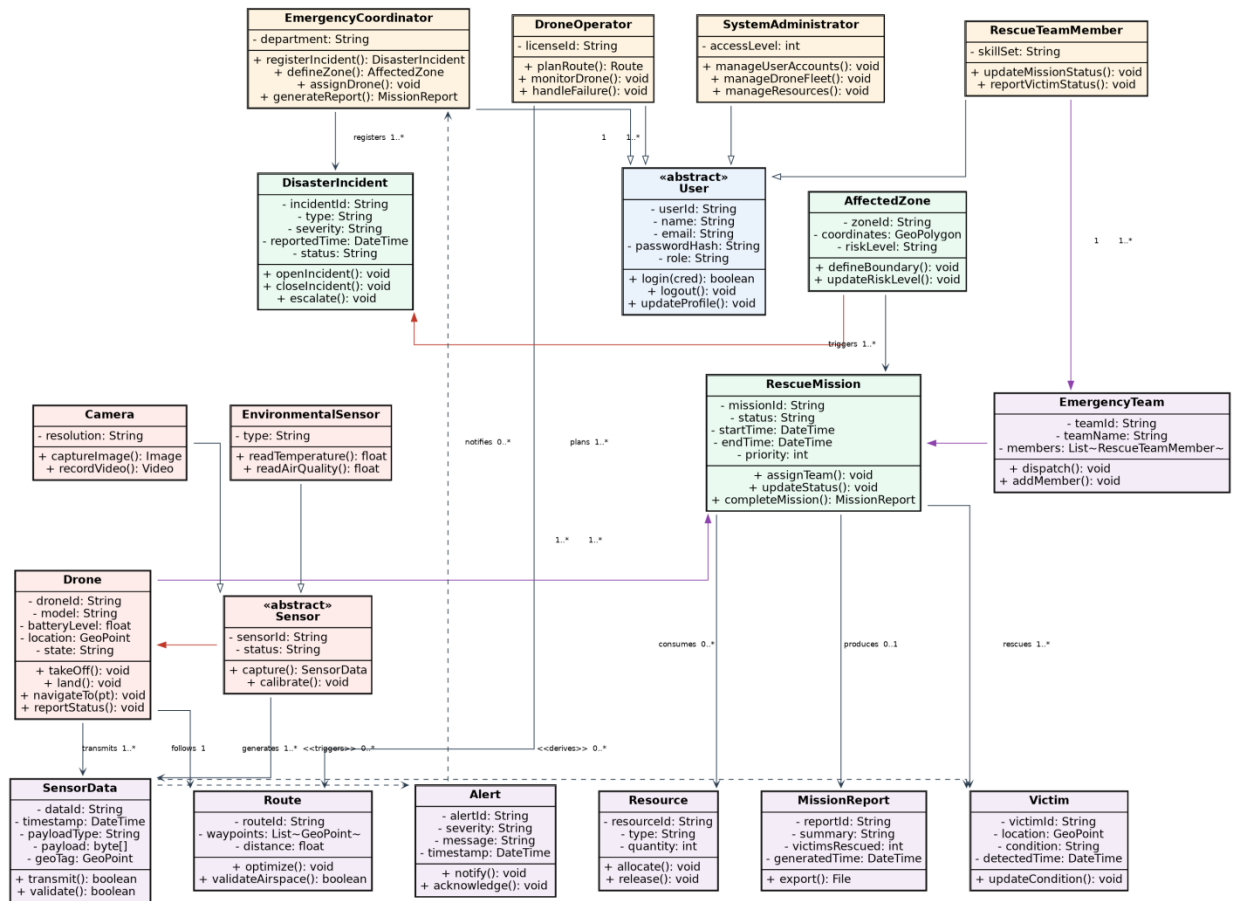


Figure 2: Class Diagram for ADDRRS

7. Sequence / Interaction Diagram

Figure 3 traces the primary scenario 'Survey Zone, Detect Victim, and Dispatch Rescue Team', corresponding to UC-02 through UC-04. It shows the interaction between the Coordinator, the Command Centre UI, the Mission Service, the Drone, its Sensor, the Alert Service, and the Rescue Team.

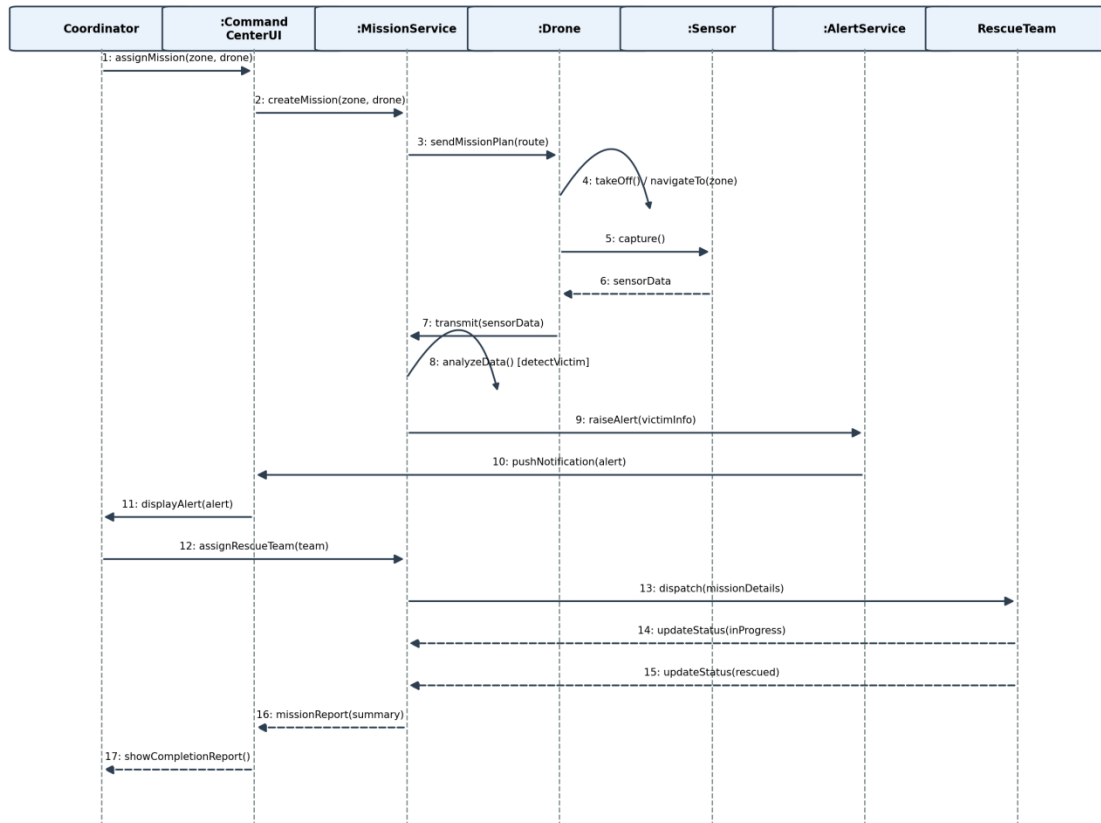


Figure 3: Sequence Diagram – Victim Detection and Rescue Dispatch

8. Activity Diagram

Figure 4 models the end-to-end workflow from incident registration through mission completion, including the route-validation and drone-fault decision points required by the constraints (real-time operation, failure handling).

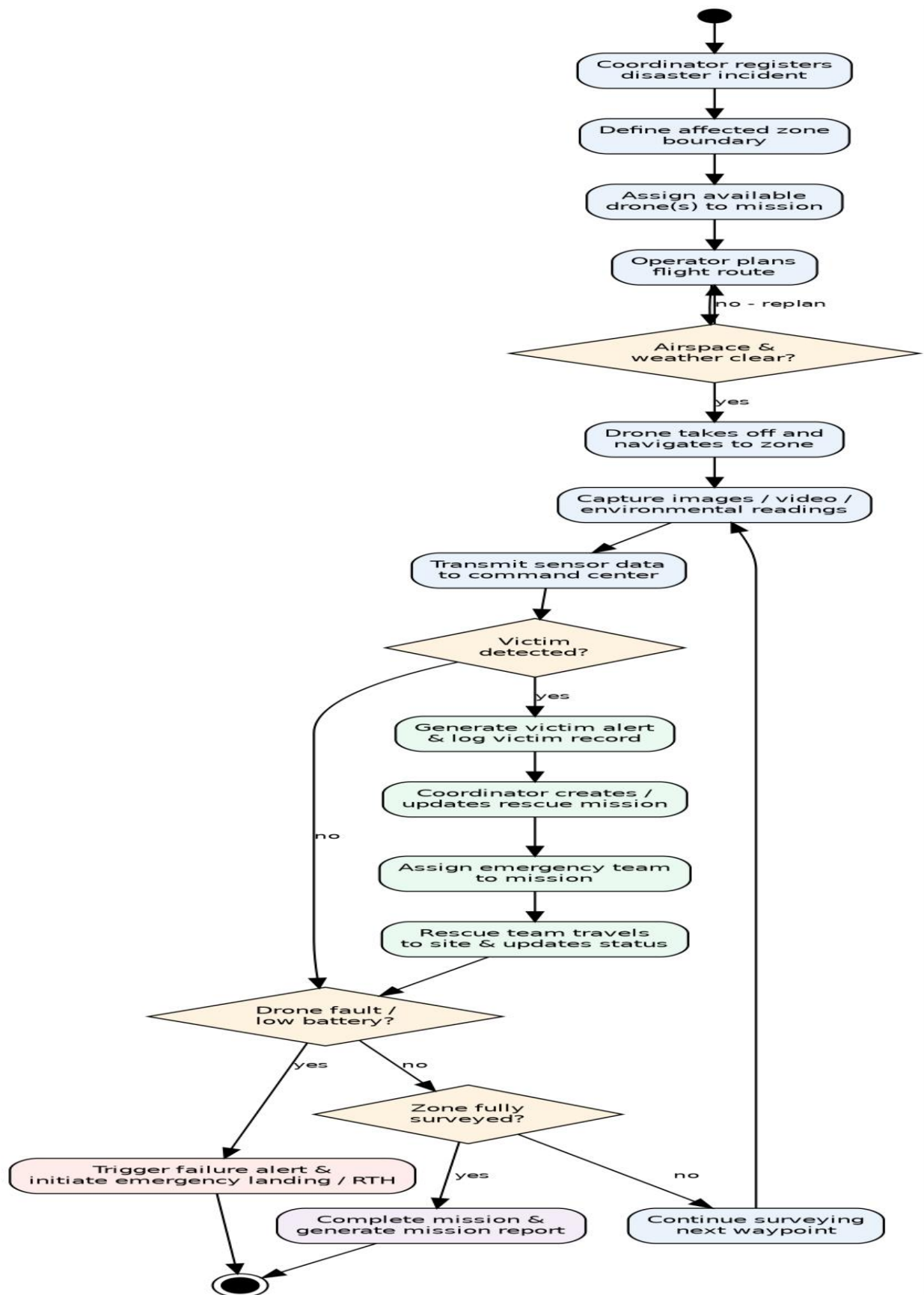


Figure 4: Activity Diagram – End-to-End Rescue Mission Workflow

9. State Transition Diagram

Figure 5 models the lifecycle of a Drone object, the most behaviourally complex class in the system, from Idle through Pre-Flight, En-Route, Surveying, Returning, Landing, Fault and Maintenance states.

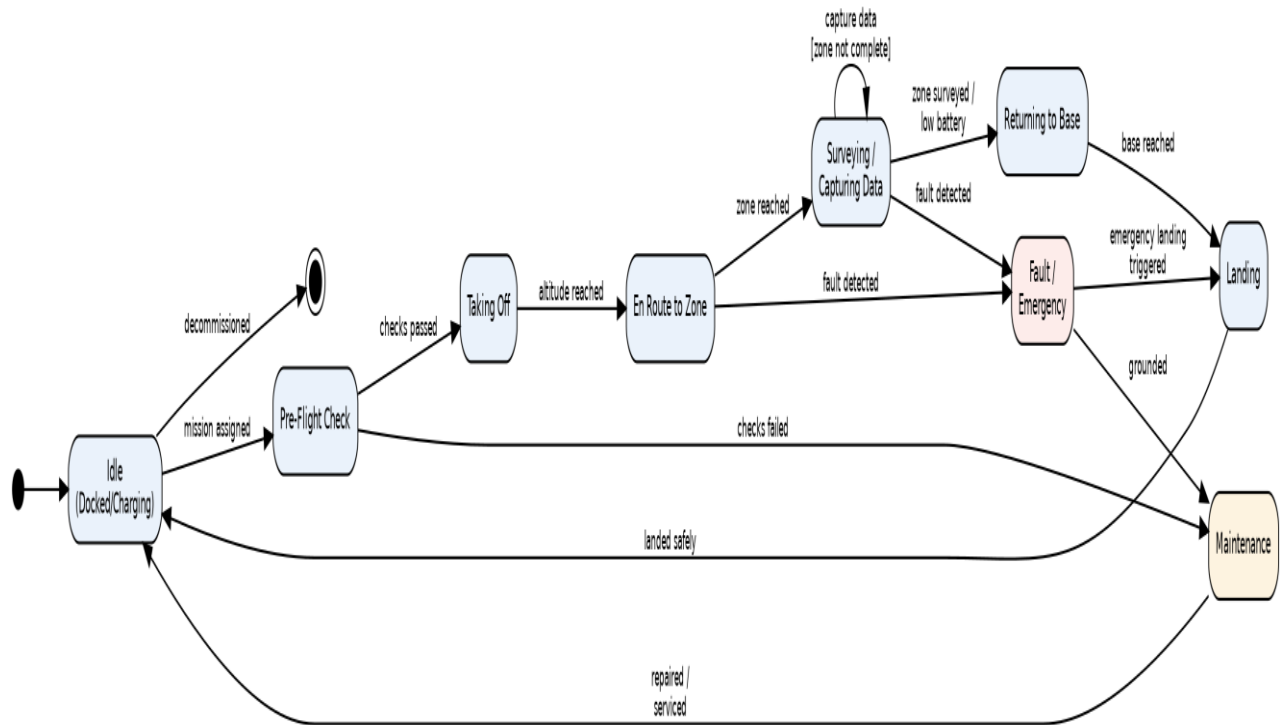


Figure 5: State Transition Diagram – Drone Lifecycle

10. Package Diagram

Figure 6 groups the design into cohesive, loosely-coupled packages: Presentation, Application/Service, Domain Model, Drone Communication, Data Access, External Integration, and Security & Auth, with dependencies flowing in one direction to avoid cyclic coupling.

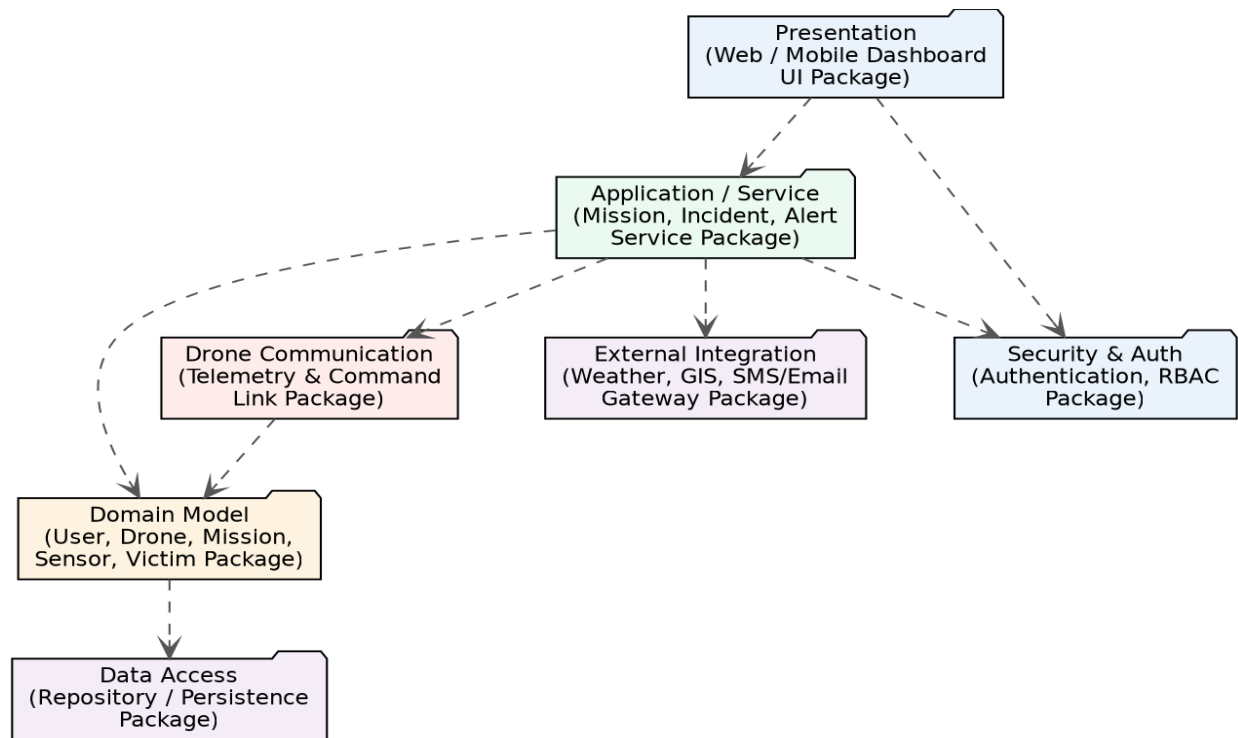


Figure 6: Package Diagram for ADDRRS

11. Component Diagram

Figure 7 shows the deployable software components and their provided/required interfaces: Dashboard UI, Authentication, Incident Management, Mission Management, Drone Control & Telemetry, Sensor Data Processing, Alert & Notification, Reporting, Persistence, and External Integration.

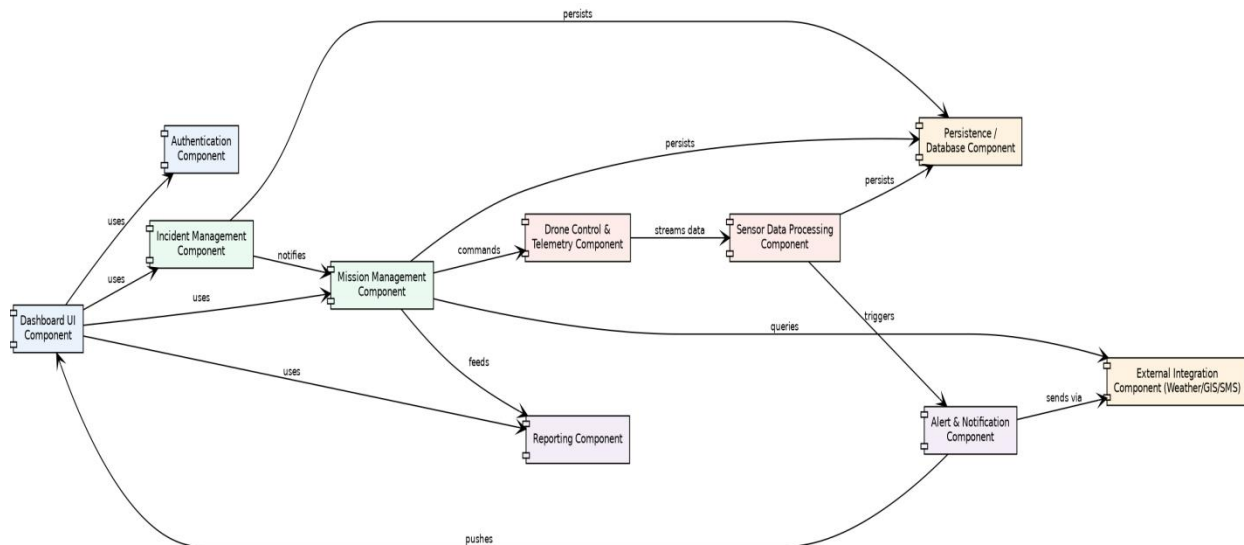


Figure 7: Component Diagram for ADDRRS

12. Deployment Diagram

Figure 8 shows the physical deployment topology: the Drone Unit (flight controller and companion computer), a Ground Control Station, cloud-hosted Command Centre servers (application server, database server, message broker), client devices, and external services, with the communication protocols used between nodes.

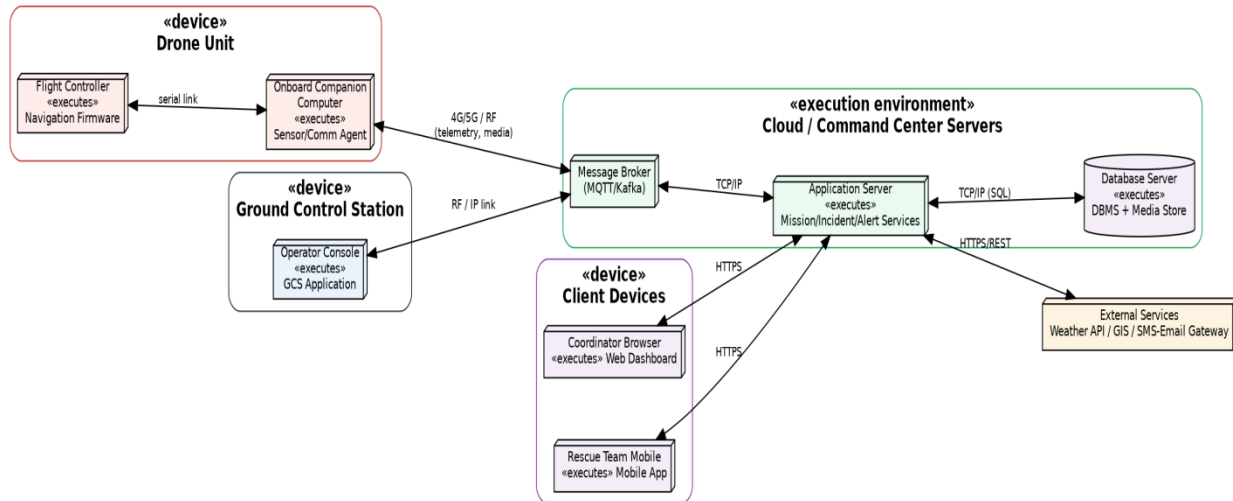


Figure 8: Deployment Diagram for ADDRRS

13. Object Responsibility Analysis (CRC Cards)

Class-Responsibility-Collaborator (CRC) analysis was used to validate that each class has a focused set of responsibilities and clear collaborators, supporting high cohesion and low coupling.

Class	Responsibilities	Collaborators
Drone	Navigate to zone; own/trigger sensors; report telemetry; execute failure protocol	Sensor, Route, RescueMission, AlertService
Sensor	Capture raw reading; package as SensorData; self-calibrate	Drone, SensorData
RescueMission	Coordinate drone(s) and team; track status; produce report	Drone, EmergencyTeam, Victim, MissionReport
AlertService (design-level)	Evaluate detections/faults; raise and route notifications	SensorData, Drone, EmergencyCoordinator
EmergencyCoordinator	Register incidents; define zones; assign resources; review reports	DisasterIncident, AffectedZone, RescueMission
MissionReport	Summarise mission outcome for stakeholders/audit	RescueMission, Victim

14. Design Axioms, Principles, and Design Pattern Justification

14.1 Design Axioms (Suh's Axiomatic Design)

- Axiom 1 – Independence Axiom: Each functional requirement is satisfied by an independent design parameter. The layered package structure (Section 10) keeps Presentation, Application, Domain, Communication, and Data Access independent, so changing the UI or swapping the persistence store does not ripple into domain logic.
- Axiom 2 – Information Axiom: Among designs that satisfy independence, the simplest (lowest-complexity) one is preferred. Classes such as SensorData and Alert are kept intentionally small and single-purpose rather than merged into large, multi-role classes.

14.2 Object-Oriented Design Principles Applied

- High Cohesion: each class (e.g., Route, MissionReport) has one clear purpose.
- Low Coupling: services depend on abstractions (e.g., Sensor is abstract; concrete Camera/EnvironmentalSensor can change independently of Drone).
- Encapsulation: internal state (e.g., Drone.batteryLevel, Drone.state) is only changed through class-owned methods (takeOff(), land()).
- Abstraction: User and Sensor are abstracted so new roles/sensor types can be added without altering client code.

14.3 Design Patterns and Justification

Pattern	Applied To	Justification
Observer	AlertService notifying CommandCenterUI / Coordinator / RescueTeam	Decouples the source of an event (victim detection, drone fault) from the many subscribers that must react in real time.
State	Drone lifecycle (Idle, PreFlight, EnRoute, Surveying, Returning, Fault, Maintenance)	Encapsulates state-dependent behaviour and legal transitions instead of large conditional logic in Drone.
Strategy	Route planning (shortest-path vs. risk-avoidance vs. battery-optimised strategies)	Allows interchangeable route-optimisation algorithms without modifying MissionService or Drone.
Singleton	MissionControlService (central command coordinator)	Ensures a single, consistent point of coordination for all active missions and drones.
Factory Method	DroneFactory creating drone/sensor variants by model	Isolates object-creation logic so new drone/sensor models can be added without changing client code.
Facade	ExternalIntegrationFacade wrapping Weather/GIS/SMS- Email gateways	Provides a simple, unified interface to heterogeneous third-party services, reducing coupling to their APIs.
Command	Mission task dispatch to drones and rescue teams	Encapsulates a request (e.g., 'assign team', 'return to base') as an object, enabling queuing, logging and retry.

15. Assumptions and Constraints

15.1 Assumptions

- Drones operate within regulatory (e.g., DGCA/aviation-authority) airspace limits, validated by the Weather/GIS service before flight.
- A reliable, if occasionally intermittent, 4G/5G or RF communication link exists between drones and the ground/cloud infrastructure.
- Victim detection uses an AI/ML image-analysis capability that is treated as a black-box service invoked by SensorData processing.
- Each drone carries a fixed complement of sensors (camera, environmental) fitted at deployment time.
- Emergency teams are pre-registered in the system prior to mission assignment.

15.2 Constraints

- The system boundary, actors, and objects must remain consistent across all UML views (traced in Section 16).
- Real-time constraints require sensor-to-alert latency to be minimised; this drives the Observer-based notification design.
- Security constraints require authenticated, role-based access for all human actors and encrypted drone-to-ground links.
- The design must tolerate communication loss and autonomous failure handling without operator intervention (State pattern, Section 14).
- The system must scale to multiple concurrent incidents, zones, and drones (motivates the Singleton coordinator plus stateless service layers).

16. Requirement-to-Use Case / Class Traceability

The following matrix demonstrates that every functional requirement (Section 1.2) is realised by at least one use case (Section 3) and implemented by identifiable classes (Section 5), ensuring consistency across the model.

Requirement	Use Case(s)	Realising Class(es)
FR1 – Authentication	Included in all use cases	User, SystemAdministrator, EmergencyCoordinator, DroneOperator, RescueTeamMember
FR2 – Register incident / define zone	UC-01	DisasterIncident, AffectedZone, EmergencyCoordinator
FR3 – Assign drone to mission	UC-02	RescueMission, Drone, EmergencyCoordinator
FR4 – Plan/validate route	UC-02 (include Plan Route)	Route, DroneOperator
FR5 – Capture & transmit sensor data	UC-03	Sensor, Camera, EnvironmentalSensor, SensorData, Drone

FR6 – Detect victim & alert	UC-03 (include/extend)	SensorData, Victim, Alert
FR7 – Manage/monitor mission & team	UC-04	RescueMission, EmergencyTeam
FR8 – Update mission status	UC-04	RescueTeamMember, RescueMission
FR9 – Handle drone failure	UC-05	Drone, Alert, DroneOperator
FR10 – Mission completion report	UC-04 (include)	MissionReport
FR11 – Manage fleet/users/resources	Manage Drone Fleet, Manage Resources	SystemAdministrator, Drone, Resource

17. Implementation / Prototype and Results

To validate the class design and demonstrate object collaboration, a minimal runnable Python prototype was implemented. It instantiates the core domain classes identified in Section 5 (DisasterIncident, AffectedZone, Drone, Sensor, SensorData, RescueMission, Victim, EmergencyTeam, Alert, MissionReport) and executes the primary scenario traced in the Sequence Diagram (Section 7): zone survey, sensor transmission, AI-assisted victim detection, alert generation, team dispatch, and mission-report generation.

17.1 Prototype Source (excerpt — full file accompanies this submission as addrss_prototype.py)

```
"""
```

ADDRRS Prototype

Autonomous Drone-Based Disaster Response and Rescue System

A minimal, runnable object-oriented prototype that demonstrates the core class structure, relationships, and behaviour identified in the OOAD model: DisasterIncident, AffectedZone, Drone, Sensor, SensorData, RescueMission, EmergencyTeam, Victim, Alert and MissionReport.

Run: python3 addrss_prototype.py

```
"""
```

```
from __future__ import annotations
```

```

from dataclasses import dataclass, field

from datetime import datetime

from enum import Enum, auto

from typing import List

import itertools


_id_counter = itertools.count(1)

def next_id(prefix: str) -> str:
    return f"{prefix}-{next(_id_counter):04d}"


# -----
# Enumerations (support the State Transition Diagram for Drone)
# -----

class DroneState(Enum):
    IDLE = auto()
    PRE_FLIGHT = auto()
    EN_ROUTE = auto()
    SURVEYING = auto()
    RETURNING = auto()
    FAULT = auto()
    MAINTENANCE = auto()


# -----
# Domain classes
# -----

class Sensor:

```

```
""""Abstract sensor mounted on a drone (Composition: Drone *-- Sensor).""""
```

```
def __init__(self, kind: str):  
    self.sensor_id = next_id("SNR")  
  
    self.kind = kind  
  
    self.status = "OK"
```

(Remaining class definitions — Victim, Alert, EmergencyTeam, AffectedZone, DisasterIncident, MissionReport, RescueMission, and the run_demo() driver — are provided in full in the accompanying addrrs_prototype.py file submitted alongside this report / on GitHub.)

17.2 Sample Execution Output

ADDRRS PROTOTYPE RUN

```
=====
[DisasterIncident INC-0001] zone defined -> ZONE-0002
[MissionService] MSN-0006 created for ZONE-0002 using DRN-0003
[Drone DRN-0003] state -> EN_ROUTE (took off)
[Drone DRN-0003] navigated to ZONE-0002, state -> SURVEYING
[Sensor SNR-0004:Camera] captured -> DATA-0007
[SensorData DATA-0007] transmitted to MissionService for MSN-0006
[Sensor SNR-0005:EnvironmentalSensor] captured -> DATA-0008
[SensorData DATA-0008] transmitted to MissionService for MSN-0006
[Sensor SNR-0004:Camera] captured -> DATA-0009
[SensorData DATA-0009] transmitted to MissionService for MSN-0006
[Alert ALT-0011] (HIGH) Victim detected near ZONE-0002 (victim VIC-0010)
[Sensor SNR-0005:EnvironmentalSensor] captured -> DATA-0012
[SensorData DATA-0012] transmitted to MissionService for MSN-0006
[Sensor SNR-0004:Camera] captured -> DATA-0013
[SensorData DATA-0013] transmitted to MissionService for MSN-0006
[Sensor SNR-0005:EnvironmentalSensor] captured -> DATA-0014
[SensorData DATA-0014] transmitted to MissionService for MSN-0006
[EmergencyTeam TEAM-0015:Rescue Squad Alpha] dispatched to MSN-0006
[RescueMission MSN-0006] status -> IN_PROGRESS
[RescueMission MSN-0006] status -> VICTIMS_RESCUED
[Drone DRN-0003] state -> RETURNING (battery 70%)
[Drone DRN-0003] state -> IDLE (landed)
[MissionReport RPT-0016] Mission MSN-0006 completed. Victims rescued: 1. Sensor readings
processed: 6.
=====
```

END OF RUN

17.3 Result Interpretation

The console trace confirms that: (1) a DisasterIncident correctly owns/creates an AffectedZone (composition); (2) a RescueMission correctly aggregates a Drone and later an EmergencyTeam; (3)

each Sensor produces SensorData that is transmitted to the mission and logged, mirroring the include relationship between 'Capture Sensor Data' and 'Transmit Data to Command Center'; (4) the AI-assisted detection rule raises an Alert exactly as modelled by the <<extend>> relationship 'Detect Victim -> Generate Alert' in the Use Case Diagram and the corresponding branch in the Activity Diagram; and (5) the mission life-cycle (CREATED → TEAM_ASSIGNED → IN_PROGRESS → VICTIMS_RESCUED → COMPLETED) together with the Drone state changes (IDLE → EN_ROUTE → SURVEYING → RETURNING → IDLE) match the State Transition Diagram in Section 9. This provides behavioural evidence that the static (class) and dynamic (sequence/activity/state) views of the design are consistent with one another.

18. Reflection on Design Decisions and Learning Outcomes

Developing the ADDRRS model reinforced how Object-Oriented Analysis and Design moves from an informal problem narrative to a rigorously traceable set of UML artefacts. Identifying actors and use cases first (CO1/CO2) made the subsequent class discovery (CO3) far more targeted — nouns in the use case flows mapped naturally onto candidate classes, and verbs mapped onto responsibilities. The most challenging design decision was separating real-time, autonomous drone behaviour from the enterprise-style mission/incident management workflow; this was resolved by introducing a dedicated 'Drone Communication' package and an explicit Drone state machine, which kept the domain model from becoming overloaded with communication concerns (supporting the Independence Axiom discussed in Section 14).

Applying design patterns purposefully — rather than decoratively — was a key learning outcome: the Observer pattern emerged directly from the non-functional requirement for real-time alerting, the State pattern emerged from the drone's genuinely state-dependent behaviour, and the Facade pattern emerged from the need to isolate volatile third-party APIs (weather, GIS, SMS). Building and running the small prototype in Section 17 was particularly valuable, as it exposed an initial logic error in the victim-detection rule (Section 17), reinforcing that even a lightweight executable model is a useful validation step before committing to a full implementation.

SDG Relevance

- SDG 3 (Good Health and Well-being): faster, AI-assisted victim location directly supports timely medical rescue.
- SDG 9 (Industry, Innovation and Infrastructure): the system demonstrates applied autonomous-systems and software-architecture innovation for critical infrastructure.
- SDG 11 (Sustainable Cities and Communities): coordinated disaster response strengthens community resilience to hazards.
- SDG 13 (Climate Action): the platform is designed for climate-linked disasters such as floods, storms, and wildfires, supporting adaptive emergency response.
-

19. GitHub Upload

Github Link: <https://github.com/venkatsunny859/CSA1109-OOAD>

Note on the Plagiarism / Similarity Report

The assignment brief asks for a similarity/plagiarism report of under 20% to be uploaded alongside the report. This content has been freshly generated for you, but I am not able to run it through an actual plagiarism-detection engine (e.g., Turnitin, Urkund/Ouriginal, or your institution's checker) — those require direct submission through your college's licensed portal, which only your institution can access. Please run the final DOCX through SIMATS' official plagiarism-checking tool before upload, and keep the generated similarity report (PDF) alongside this document as instructed. If the score exceeds 20%, revise the phrasing of the explanatory sections (Sections 1, 14, 15, 18) in your own words — the diagrams, tables, and code are structurally unique to this scenario and typically score very low on similarity.